

Syllabus: DBMS Unit – 3

Unit 3 : Data Base Design & Normalization

Functional dependencies, normal forms, first, second, third normal forms, BCNF, inclusion dependence, loss less join decompositions, normalization using FD, MVD, and JDs, alternative approaches to database design

Priyanka Dhanraj

Functional dependencies

- **Functional Dependency (FD)** determines the **relation of one attribute to another attribute** in a database management system (DBMS) system. Functional dependency **helps you to maintain the quality of data in the database.**
- The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table.
- A functional dependency is denoted by an arrow $\rightarrow$. The functional dependency is represented by $A \rightarrow B$. **or say B is functionally dependent on A. The left side of FD is known as a determinant, the right side of the production is known as a dependent.**
- Functional Dependency **plays a vital role to find the difference between good and bad database design.**
- A function dependency $A \rightarrow B$ means for all instances of a particular value of A, there is the same value of B. For example in table $A \rightarrow B$ is true, but $B \rightarrow A$ is not true as there are different values of A for $B = 3$.

A	B
1	3
2	3
4	0
1	3
4	0

Functional dependencies

- For example: Suppose we have a **student table with attributes: Stu_Id, Stu_Name, Stu_Age**. Here **Stu_Id** attribute uniquely identifies the **Stu_Name** attribute of student table because if we know the student id we can tell the student name associated with it. This is known as **functional dependency** and can be written as **Stu_Id → Stu_Name** or in words we can say **Stu_Name is functionally dependent on Stu_Id**.
- A simple example of single value functional dependency is when **Roll_Number** is the primary key of an entity and **Student_Name** is some single valued attribute of the entity. Then, **Roll_Number → Student_Name**.

Roll_Number	Student_Name	Student_Address
011	Jayesh Umre	Burhanpur
012	Kunal Batra	Burhanpur
013	Nilesh Nimbhorkar	Ichchapur
014	Aryan Jagdale	Ujjain

Functional dependencies

- A functional dependency $P \rightarrow Q$ is full functional dependency if removal of any attribute A from P means that the dependency does not hold any more. For Example $\{\text{Roll_Number}, \text{Subject_Name}\} \rightarrow \text{Paper_Hour}$ Since neither $\text{Roll_Number} \rightarrow \text{Paper_Hour}$ nor $\text{Subject_Name} \rightarrow \text{Paper_Hour}$ hold.
- A Functional Dependency in which one or more non key attributes are functionally depending on a part of the primary key is called partial functional dependency. For Example $\{\text{Roll_Number}, \text{Subject_Name}\} \rightarrow \text{Student_Name}$ is not a full FD. Since $\text{Roll_Number} \rightarrow \text{Student_Name}$ also hold.
- Normalization should be part of the database design process. However, it is difficult to separate the normalization process from the ER modelling process so the two techniques should be used concurrently.
- Use an entity relation diagram (ERD) to provide the big picture, or macro view, of an organization's data requirements and operations. This is created through an iterative process that involves identifying relevant entities, their attributes and their relationships.
- Normalization procedure focuses on characteristics of specific entities and represents the micro view of entities within the ERD.

Rules of Functional Dependencies (Inference Rules)

The **Armstrong's axioms** are the **basic inference rule**. Armstrong's axioms are **used to conclude functional dependencies on a relational database**. The most important rules for Functional Dependency:

- **Reflexive rule** – If **A is a set of attributes and B is_subset_of A**, then A holds a value of B ($A \rightarrow B$).
- **Transitivity rule**: This rule is very much similar to the transitive rule in algebra if **A $\rightarrow$ B holds and B $\rightarrow$ C holds, then A $\rightarrow$ C also holds**. A $\rightarrow$ B is called as functionally that determines B.
- **Augmentation rule**: When **A $\rightarrow$ B holds, and C is attribute set, then AC $\rightarrow$ BC also holds**. That is **adding attributes which do not change the basic dependencies (partial dependency)**.
- **Decomposition**- If **A $\rightarrow$ BC**, then **A $\rightarrow$ B and A $\rightarrow$ C** always holds.
- **Composition/ Pseudo transitive Rule** - If **A $\rightarrow$ B and C $\rightarrow$ D**, then **AC $\rightarrow$ BD** always holds (If **X $\rightarrow$ Y** and **YZ $\rightarrow$ W** then **XZ $\rightarrow$ W**).
- **Additive/ Union**- If **A $\rightarrow$ B and A $\rightarrow$ C**, then **A $\rightarrow$ BC** always holds.

Types of Functional Dependencies

1. Multivalued dependency
2. Trivial functional dependency
3. Non-trivial functional dependency
4. Transitive dependency

Priyanka Dhanraj

Multivalued dependency

- Multivalued dependency : Multivalued dependency occurs when there are more than one independent multivalued attributes in a table. A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.
- MVD or say Multivalued Dependency defines that for one value of attribute 'x' multiple values of attribute 'y' be existent.
- for a relation $R(X, Y, Z)$, if there exists a multivalued dependency between X and Y , so Y and Z should have to be independent of each other.
- For example: Consider a bike manufacture company, which produces two colors (Black and Red) in each model every year.
- Here columns `manuf_year` and `color` are independent of each other and dependent on `bike_model`. In this case these two columns are said to be multivalued dependent on `bike_model`. These dependencies can be represented like this: Breaking the relation into two relation.

`bike_model ->> manuf_year`

`bike_model ->> color`

Car-model Name	Manufacture year	Color
Maruti Suzuki Swift	2007	Black
Maruti Suzuki Swift	2007	Red
Maruti Suzuki Swift	2008	Black
Maruti Suzuki Swift	2008	Red

Trivial functional dependency

- The **dependency of an attribute on a set of attributes** is known as trivial functional dependency if the set of attributes includes that attribute. **If RHS of a functional dependency is a subset of LHS, then it is called as a trivial functional dependency.**
- Symbolically: $A \rightarrow B$ is trivial functional dependency if B is a subset of A .
- For example: Consider a table with **two columns Student_id and Student_Name**. **$\{Student_Id, Student_Name\} \rightarrow Student_Id$ is a trivial functional dependency as $Student_Id$ is a subset of $\{Student_Id, Student_Name\}$.** **That makes sense because if we know the values of $Student_Id$ and $Student_Name$ then the value of $Student_Id$ can be uniquely determined.**
- Also, $Student_Id \rightarrow Student_Id$ & $Student_Name \rightarrow Student_Name$ are trivial dependencies too.
- The examples of trivial functional dependencies are-
 $AB \rightarrow A$, $AB \rightarrow B$, $AB \rightarrow AB$

Non-Trivial functional dependency

- If a functional dependency $X \rightarrow Y$ holds true where Y is not a subset of X then this dependency is called non trivial Functional dependency. If there exists at least one attribute in the RHS of a functional dependency that is not a part of LHS, then it is called as a non-trivial functional dependency. Example $AB \rightarrow BC$, $AB \rightarrow CD$.
- For example: An employee table with three attributes: emp_id, emp_name, emp_address.
- The following functional dependencies are non-trivial:
 $emp_id \rightarrow emp_name$ (emp_name is not a subset of emp_id)
 $emp_id \rightarrow emp_address$ (emp_address is not a subset of emp_id)
- On the other hand, the following dependencies are trivial:
 $\{emp_id, emp_name\} \rightarrow emp_name$ [emp_name is a subset of {emp_id, emp_name}]
- **Completely non trivial FD:** If a FD $X \rightarrow Y$ holds true where $X \cap Y$ is null then this dependency is said to be completely non trivial function dependency.

Transitive dependency

- A functional dependency is said to be transitive if it is indirectly formed by two functional dependencies.
- For e.g. $X \rightarrow Z$ is a transitive dependency if the following three functional dependencies hold true:
 $X \rightarrow Y$, $Y \rightarrow Z$, $Y \not\rightarrow X$
- Note: A transitive dependency can only occur in a relation of three or more attributes. This dependency helps us normalizing the database in 3NF (3rd Normal Form).

- Example:

Book	Author	Author_age
Game of Thrones	George R. R. Martin	66
Harry Potter	J. K. Rowling	49
Dying of the Light	George R. R. Martin	66

Transitive dependency

$\{Book\} \rightarrow \{Author\}$ (if we know the book, we know the author name)

$\{Author\}$ does not $\rightarrow \{Book\}$

$\{Author\} \rightarrow \{Author_age\}$

- Therefore as per the rule of **transitive dependency**: $\{Book\} \rightarrow \{Author_age\}$ should hold, that makes sense because **if we know the book name we can know the author's age.**

Advantages of Functional Dependency

- Functional Dependency **avoids data redundancy**. Therefore same data do not repeat at multiple locations in that database.
- It helps you to **maintain the quality of data** in the database
- It helps you to **defined meanings and constraints of databases**
- It helps you **to identify bad designs**
- It helps you to **find the facts regarding the database design**

Closure of an Attribute Set

- The set of all those attributes which can be functionally determined from an attribute set is called as a closure of that attribute set. Closure of attribute set $\{X\}$ is denoted as $\{X\}^+$

Steps to Find Closure of an Attribute Set-

- Step-01: Add the attributes contained in the attribute set for which closure is being calculated to the result set.
- Step-02: Recursively add the attributes to the result set which can be functionally determined from the attributes already contained in the result set.

Example- Consider a relation $R (A , B , C , D , E , F , G)$ with the functional dependencies-

$A \rightarrow BC , BC \rightarrow DE , D \rightarrow F , CF \rightarrow G$

Closure of an Attribute Set

$A \rightarrow BC, BC \rightarrow DE, D \rightarrow F, CF \rightarrow G$

- Closure of attribute A : $A^+ = \{ A \},$

$= \{ A, B, C \} (\text{Using } A \rightarrow BC)$

$= \{ A, B, C, D, E \} (\text{Using } BC \rightarrow DE)$

$= \{ A, B, C, D, E, F \} (\text{Using } D \rightarrow F)$

$= \{ A, B, C, D, E, F, G \} (\text{Using } CF \rightarrow G)$

Thus, $A^+ = \{ A, B, C, D, E, F, G \}$

Priyanka Dhanraj

Closure of an Attribute Set

$A \rightarrow BC, BC \rightarrow DE, D \rightarrow F, CF \rightarrow G$

- Closure of attribute D : $D^+ = \{ D \} = \{ D, F \}$ (Using $D \rightarrow F$)

We can not determine any other attribute using attributes D and F contained in the result set. Thus,

$D^+ = \{ D, F \}$

Closure of attribute set {B, C}- $\{ B, C \}^+ = \{ B, C \} = \{ B, C, D, E \}$ (Using $BC \rightarrow DE$)

$= \{ B, C, D, E, F \}$ (Using $D \rightarrow F$) $= \{ B, C, D, E, F, G \}$ (Using $CF \rightarrow G$)

Thus,

$\{ B, C \}^+ = \{ B, C, D, E, F, G \}$

Closure of an Attribute Set

Problem- Consider the given functional dependencies-

$AB \rightarrow CD$, $AF \rightarrow D$, $DE \rightarrow F$, $C \rightarrow G$, $F \rightarrow E$, $G \rightarrow A$

Which of the following options is false?

(A) $\{CF\}^+ = \{A, C, D, E, F, G\}$

(B) $\{BG\}^+ = \{A, B, C, D, G\}$

(C) $\{AF\}^+ = \{A, C, D, E, F, G\}$

(D) $\{AB\}^+ = \{A, C, D, F, G\}$

Priyanka Dhanraj

Closure of an Attribute Set

Solution-Let us check each option one by one-

Option-(A): $\{CF\}^+ = \{C, F\}, = \{C, F, G\}$ (Using $C \rightarrow G$), $= \{C, E, F, G\}$ (Using $F \rightarrow E$)
 $= \{A, C, E, E, F\}$ (Using $G \rightarrow A$), $= \{A, C, D, E, F, G\}$ (Using $AF \rightarrow D$)

Since, our obtained result set is same as the given result set, so, it means it is correctly given.

Option-(B): $\{BG\}^+ = \{B, G\}, = \{A, B, G\}$ (Using $G \rightarrow A$), $= \{A, B, C, D, G\}$ (Using $AB \rightarrow CD$)

Since, our obtained result set is same as the given result set, so, it means it is correctly given.

Option-(C): $\{AF\}^+ = \{A, F\}, = \{A, D, F\}$ (Using $AF \rightarrow D$), $= \{A, D, E, F\}$ (Using $F \rightarrow E$)

So, it means it is not correctly given.

Option-(D): $\{AB\}^+ = \{A, B\}, = \{A, B, C, D\}$ (Using $AB \rightarrow CD$), $= \{A, B, C, D, G\}$ (Using $C \rightarrow G$)

So, it means it is not correctly given. Thus, Option (C) and Option (D) are correct.

Super Key- Finding the Keys Using Closure

Super Key- If the closure result of an attribute set contains all the attributes of the relation, then that attribute set is called as a super key of that relation. “The closure of a super key is the entire relation schema.” It is not at all compulsory that all super keys are candidate keys.

Example- The closure of attribute A is the entire relation schema. Thus, attribute A is a super key for that relation.

Example- Consider a relation R (A , B , C , D , E , F , G) with the functional dependencies-

$A \rightarrow BC$, $BC \rightarrow DE$, $D \rightarrow F$, $CF \rightarrow G$

- Closure of attribute A : $A^+ = \{ A \}$, $= \{ A , B , C \}$ (Using $A \rightarrow BC$)
 $= \{ A , B , C , D , E \}$ (Using $BC \rightarrow DE$) , $= \{ A , B , C , D , E , F \}$ (Using $D \rightarrow F$)
 $= \{ A , B , C , D , E , F , G \}$ (Using $CF \rightarrow G$) , Thus, $A^+ = \{ A , B , C , D , E , F , G \}$

Candidate Key- Finding the Keys Using Closure

Candidate Key- Candidate Key is minimal set of attributes of a relation which can be used to identify a tuple uniquely. A candidate key may or may not be a primary key but all candidate keys are super keys. **If there exists no subset of an attribute set whose closure contains all the attributes of the relation, then that attribute set is called as a candidate key of that relation.** Prime attribute is the attribute which is the part of the candidate key. Non -prime attribute is the attribute which is not the part of the candidate key.

Priyanka Dhanraj

Finding Candidate Keys- We can determine the candidate keys of a given relation using the following Step-01: Determine all essential attributes of the given relation. Essential attributes are those attributes which are **not present on RHS** of any functional dependency. **Essential attributes are always a part of every candidate key.** This is because they can not be determined by other attributes.

Example : Let $R(A, B, C, D, E, F)$ be a relation scheme with the following functional dependencies-

$A \rightarrow B, C \rightarrow D, D \rightarrow E$ So, essential attributes are- A, C and F.

Candidate Key- Finding the Keys Using Closure

Step-02: The remaining attributes of the relation are non-essential attributes.

This is because they can be determined by using essential attributes.

Now, following two cases are possible-

Case-01: If all essential attributes together can determine all remaining non-essential attributes, then- The combination of essential attributes is the candidate key. It is the only possible candidate key.

Case-02: If all essential attributes together can not determine all remaining non-essential attributes, then- The set of essential attributes and some non-essential attributes will be the candidate key(s). In this case, multiple candidate keys are possible.

To find the candidate keys, we check different combinations of essential and non-essential attributes.

Problem on Finding the Keys Using Closure

Problem-01: Let $R = (A, B, C, D, E, F)$ be a relation scheme with the following dependencies-

$C \rightarrow F, E \rightarrow A, EC \rightarrow D, A \rightarrow B$, Which of the following is a key for R ?

1. CD
2. EC
3. AE
4. AC

Priyanka Dhanraj

Also, determine the total number of candidate keys.

Step-01: Determine all essential attributes of the given relation.

Essential attributes of the relation are- C and E .

So, attributes C and E will definitely be a part of every candidate key.

Problem on Finding the Keys Using Closure

Step-02: Now, We will check if the essential attributes together can determine all remaining non-essential attributes.

To check, we find the closure of CE. So, we have- $\{CE\}^+ = \{C, E\}, = \{C, E, F\}$ (Using $C \rightarrow F$)

$= \{A, C, E, F\}$ (Using $E \rightarrow A$), $= \{A, C, D, E, F\}$ (Using $EC \rightarrow D$)

$= \{A, B, C, D, E, F\}$ (Using $A \rightarrow B$)

Priyanka Dhanraj

We conclude that CE can determine all the attributes of the given relation.

So, CE is the only possible candidate key of the relation. Thus, Option (B) is correct.

Total Number of Candidate Keys-

Only one candidate key CE is possible.

Problem on Finding the Keys Using Closure

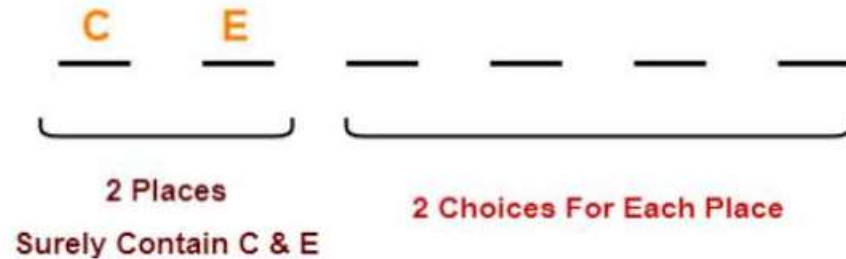
Total Number of Super Keys-There are total 6 attributes in the given relation of which-

There are 2 essential attributes- C and E.

Remaining 4 attributes are non-essential attributes.

Essential attributes will be definitely present in every key.

Non-essential attributes may or may not be taken in every super key.



So, number of super keys possible = $2 \times 2 \times 2 \times 2 = 16$.

Thus, total number of super keys possible = 16.

Problem on Finding the Keys Using Closure

Problem-02: Consider the relation scheme $R(A, B, C, D, E, H)$ and the set of functional dependencies-

$A \rightarrow B$, $BC \rightarrow D$, $E \rightarrow C$, $D \rightarrow A$

What are the candidate keys of R ?

1. AE, BE
2. AE, BE, DE
3. AEH, BEH, BCH
4. AEH, BEH, DEH

Priyanka Dhanraj

Solution-We will find candidate keys of the given relation in the following steps-

Step-01: Determine all essential attributes of the given relation. Essential attributes of the relation are- E and H. So, attributes E and H will definitely be a part of every candidate key.

The only possible option is (D). Thus, Option (D) is correct.

Membership Test

In the membership test, we check a given functional dependency is a member (set) of functional dependency set or not.

To check whether $X \rightarrow Y$ (FDs) is a member of the Functional Dependency set or not. We check by closer that $X \rightarrow Y$ is present in the Functional Dependency set or not.

Let $\{A \rightarrow B, B \rightarrow C\} \models A \rightarrow C$ that means $A \rightarrow C$ is present in this FDs $\{A \rightarrow B, B \rightarrow C\}$.

Q. Check following membership valid or not

1. $\{X \rightarrow Y, Y \supseteq Z\} \models X \rightarrow Z$
2. $\{XY \rightarrow Z, Y \rightarrow W\} \models WX \rightarrow Z$
3. $\{XY \rightarrow Z, Z \rightarrow X\} \models Z \rightarrow Y$
4. $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow YZ$
5. $\{XY \rightarrow Z, Z \rightarrow W\} \models X \rightarrow W$

Membership Test

Solutions:

1. $\{X \rightarrow Y, Y \supseteq Z\}$ and $Y \supseteq Z$ the $Y \rightarrow Z$ (**Trivial FD**)

$$\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow Z$$

$X^+ = XYZ$ so, **it is a member.**

2. $\{XY \rightarrow Z, Y \rightarrow W\} \models WX \rightarrow Z$

$(WX)^+ = WX$ so, $WX \rightarrow Z$ is **not a member.**

3. $\{XY \rightarrow Z, Z \rightarrow X\} \models Z \rightarrow Y$

$Z^+ = ZX$ so, $Z \rightarrow Y$ is **not a member.**

4. $\{X \rightarrow Y, Y \rightarrow Z\} \models X \rightarrow YZ$

$$X^+ = XYZ = 2^3 = 8$$

Priyanka Dhanraj
From 8 FDs we get one FD $X \rightarrow YZ$ so, $X \rightarrow YZ$ **is a member.**

5. $\{XY \rightarrow Z, Z \rightarrow W\} \models X \rightarrow W$

$X^+ = X$ so, $X \rightarrow W$ is **not a member.**

Equivalence of Two Sets of Functional Dependencies

In DBMS, Two different sets of functional dependencies for a given relation may or may not be equivalent. If F and G are the two sets of functional dependencies, then following 3 cases are possible-

1. Case-01: F covers G ($F \supseteq G$)
2. Case-02: G covers F ($G \supseteq F$)
3. Case-03: Both F and G cover each other ($F = G$)

Priyanka Dhanraj

Equivalence of Two Sets of Functional Dependencies

Case-01: Determining Whether F Covers G : Following steps are followed to determine whether F covers G or not-

Step-01: Take the functional dependencies of set G into consideration. For each functional dependency $X \rightarrow Y$, find the closure of X using the functional dependencies of set G.

Step-02: Take the functional dependencies of set F into consideration. For each functional dependency $X \rightarrow Y$, find the closure of X using the functional dependencies of set F.

Step-03: Compare the results of Step-01 and Step-02.

If the functional dependencies of set F has determined all those attributes that were determined by the functional dependencies of set G, then it means F covers G. Thus, we conclude F covers G ($F \supseteq G$) otherwise not.

Equivalence of Two Sets of Functional Dependencies

Case-02: Determining Whether G Covers F- Following steps are followed to determine whether G covers F or not-

Step-01: Take the functional dependencies of set F into consideration. For each functional dependency $X \rightarrow Y$, find the closure of X using the functional dependencies of set F.

Step-02: Take the functional dependencies of set G into consideration. For each functional dependency $X \rightarrow Y$, find the closure of X using the functional dependencies of set G.

Step-03: Compare the results of Step-01 and Step-02.

If the functional dependencies of set G has determined all those attributes that were determined by the functional dependencies of set F, then it means G covers F.

Thus, we conclude G covers F ($G \supseteq F$) otherwise not.

Equivalence of Two Sets of Functional Dependencies

Case-03: Determining Whether Both F and G Cover Each Other-

If F covers G and G covers F, then both F and G cover each other.

Thus, if both the above cases hold true, we conclude both F and G cover each other ($F = G$).

Problem-A relation R (A , C , D , E , H) is having two functional dependencies sets F and G as shown-

Set F- $A \rightarrow C$, $AC \rightarrow D$, $E \rightarrow AD$, $E \rightarrow H$

Set G- $A \rightarrow CD$, $E \rightarrow AH$

Which of the following holds true?

(A) $G \supseteq F$ (G covers F)

(B) $F \supseteq G$ (F covers G)

(C) $F = G$

(D) All of the above

Equivalence of Two Sets of Functional Dependencies

Solution- Determining whether G covers F- **Relation R (A , C , D , E , H) -**

Set F- $A \rightarrow C$, $AC \rightarrow D$, $E \rightarrow AD$, $E \rightarrow H$ and Set G- $A \rightarrow CD$, $E \rightarrow AH$

Step-01: $(A)^+ = \{ A , C , D \}$ // closure of left side of $A \rightarrow C$ using set F

$(AC)^+ = \{ A , C , D \}$ // closure of left side of $AC \rightarrow D$ using set F

$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of $E \rightarrow AD$ and $E \rightarrow H$ using set F

Step-02: $(A)^+ = \{ A , C , D \}$ // closure of left side of $A \rightarrow C$ using set G

$(AC)^+ = \{ A , C , D \}$ // closure of left side of $AC \rightarrow D$ using set G

$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of $E \rightarrow AD$ and $E \rightarrow H$ using set G

Step-03: Comparing the results of Step-01 and Step-02, we find-

Functional dependencies of set G can determine all the attributes which have been determined by the functional dependencies of set F.

Thus, we conclude G covers F i.e. $G \supseteq F$.

Equivalence of Two Sets of Functional Dependencies

Determining whether F covers G-Relation R (A , C , D , E , H) -

Set F- $A \rightarrow C$, $AC \rightarrow D$, $E \rightarrow AD$, $E \rightarrow H$ and Set G- $A \rightarrow CD$, $E \rightarrow AH$

Step-01: $(A)^+ = \{ A , C , D \}$ // closure of left side of $A \rightarrow CD$ using set G

$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of $E \rightarrow AH$ using set G

Step-02: $(A)^+ = \{ A , C , D \}$ // closure of left side of $A \rightarrow CD$ using set F

$(E)^+ = \{ A , C , D , E , H \}$ // closure of left side of $E \rightarrow AH$ using set F

Step-03: Comparing the results of Step-01 and Step-02, we find-

Functional dependencies of set F can determine all the attributes which have been determined by the functional dependencies of set G. Thus, we conclude F covers G i.e. $F \supseteq G$.

Determining whether both F and G cover each other- From Step-01, we conclude F covers G.

From Step-02, we conclude G covers F. Thus, we conclude both F and G cover each other i.e. $F = G$.

Thus, Option (D) is correct.

Canonical Cover in DBMS

In DBMS, A canonical cover is a simplified and reduced version of the given set of functional dependencies. Since it is a reduced version, it is also called as Irreducible set.

Characteristics-

- Canonical cover is free from all the extraneous functional dependencies.
- The closure of canonical cover is same as that of the given set of functional dependencies.
- Canonical cover is not unique and may be more than one for a given set of functional dependencies.

Need-

- Working with the set containing extraneous functional dependencies increases the computation time.
- Therefore, the given set is reduced by eliminating the useless functional dependencies.
- This reduces the computation time and working with the irreducible set becomes easier.

Canonical Cover in DBMS

Steps To Find Canonical Cover- with example.

Problem- The following functional dependencies hold true for the relational scheme $R (W , X , Y , Z)$

$X \rightarrow W, WZ \rightarrow XY, Y \rightarrow WXZ$ Write the irreducible equivalent for this set of functional dependencies.

Solution-

Step-01: Write all the functional dependencies such that each contains exactly one attribute on its right side- $X \rightarrow W, WZ \rightarrow X, WZ \rightarrow Y, Y \rightarrow W, Y \rightarrow X, Y \rightarrow Z$.

Step-02: Check the essentiality of each functional dependency one by one.

For $X \rightarrow W$: Considering $X \rightarrow W, (X)^+ = \{ X , W \}$ Ignoring $X \rightarrow W, (X)^+ = \{ X \}$ Now, Clearly, the two results are different. Thus, we conclude that $X \rightarrow W$ is essential and can not be eliminated.

For $WZ \rightarrow X$: Considering $WZ \rightarrow X, (WZ)^+ = \{ W , X , Y , Z \}$

Ignoring $WZ \rightarrow X, (WZ)^+ = \{ W , X , Y , Z \}$ Now, Clearly, the two results are same.

Thus, we conclude that $WZ \rightarrow X$ is non-essential and can be eliminated.

Canonical Cover in DBMS

Before : $X \rightarrow W$, $WZ \rightarrow X$, $WZ \rightarrow Y$, $Y \rightarrow W$, $Y \rightarrow X$, $Y \rightarrow Z$.

Eliminating $WZ \rightarrow X$, our set of functional dependencies reduces to-

$X \rightarrow W$, $WZ \rightarrow Y$, $Y \rightarrow W$, $Y \rightarrow X$, $Y \rightarrow Z$

Now, we will consider this reduced set in further checks.

For $WZ \rightarrow Y$: Considering $WZ \rightarrow Y$, $(WZ)^+ = \{W, X, Y, Z\}$

Ignoring $WZ \rightarrow Y$, $(WZ)^+ = \{W, Z\}$ Now, Clearly, the two results are different.

Thus, we conclude that $WZ \rightarrow Y$ is essential and can not be eliminated.

For $Y \rightarrow W$: Considering $Y \rightarrow W$, $(Y)^+ = \{W, X, Y, Z\}$ Ignoring $Y \rightarrow W$, $(Y)^+ = \{W, X, Y, Z\}$ Now,

Clearly, the two results are same.

Thus, we conclude that $Y \rightarrow W$ is non-essential and can be eliminated.

Canonical Cover in DBMS

Before : $X \rightarrow W$, $WZ \rightarrow Y$, $Y \rightarrow W$, $Y \rightarrow X$, $Y \rightarrow Z$

Eliminating $Y \rightarrow W$, our set of functional dependencies reduces to-

$X \rightarrow W$, $WZ \rightarrow Y$, $Y \rightarrow X$, $Y \rightarrow Z$

For $Y \rightarrow X$: Considering $Y \rightarrow X$, $(Y)^+ = \{ W, X, Y, Z \}$

Ignoring $Y \rightarrow X$, $(Y)^+ = \{ Y, Z \}$ Now, Clearly, the two results are different.

Thus, we conclude that $Y \rightarrow X$ is essential and can not be eliminated.

For $Y \rightarrow Z$: Considering $Y \rightarrow Z$, $(Y)^+ = \{ W, X, Y, Z \}$ Ignoring $Y \rightarrow Z$, $(Y)^+ = \{ W, X, Y \}$ Now, Clearly, the two results are different. Thus, we conclude that $Y \rightarrow Z$ is essential and can not be eliminated.

From here, our essential functional dependencies are-

$X \rightarrow W$, $WZ \rightarrow Y$, $Y \rightarrow X$, $Y \rightarrow Z$

Canonical Cover in DBMS

Step-03: Consider the functional dependencies having more than one attribute on their left side.

Check if their left side can be reduced.

In our set, Only $WZ \rightarrow Y$ contains more than one attribute on its left side.

Considering $WZ \rightarrow Y$, $(WZ)^+ = \{ W, X, Y, Z \}$ Now, Consider all the possible subsets of WZ .

Check if the closure result of any subset matches to the closure result of WZ .

$(W)^+ = \{ W \}$, $(Z)^+ = \{ Z \}$

Clearly, None of the subsets have the same closure result same as that of the entire left side.

Thus, we conclude that we can not write $WZ \rightarrow Y$ as $W \rightarrow Y$ or $Z \rightarrow Y$.

Thus, set of functional dependencies obtained in step-02 is the canonical cover.

Finally, the canonical cover is- $X \rightarrow W, WZ \rightarrow Y, Y \rightarrow X, Y \rightarrow Z$ Canonical Cover.

Decomposition of a Relation

- When a relation in the relational model is not in appropriate normal form then the decomposition of a relation is required.
- The process of breaking up or dividing a single relation into two or more sub relations is called as decomposition of a relation.
- If the relation has no proper decomposition, then it may lead to problems like loss of information.
- Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.
- The following two properties must be followed when decomposing a given relation
 1. Lossless decomposition
 2. Dependency Preservation

Priyanka Dhanraj

Properties of Decomposition

The following two properties must be followed when decomposing a given relation

1. Lossless decomposition- Lossless decomposition ensures-

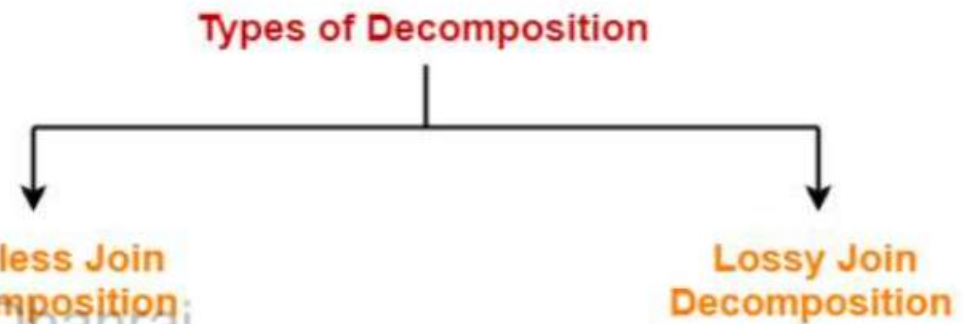
- No information is lost from the original relation during decomposition.
- When the sub relations are joined back, the same relation is obtained that was decomposed.
- Every decomposition must always be lossless.

2. Dependency Preservation- Dependency preservation ensures-

- None of the functional dependencies that holds on the original relation are lost.
- The sub relations still hold or satisfy the functional dependencies of the original relation.
- If a relation R is decomposed into relation R1 and R2, then the dependencies of R either must be a part of R1 or R2 or must be derivable from the combination of functional dependencies of R1 and R2.
- For example, suppose there is a relation R (A, B, C, D) with functional dependency set (A→BC). The relational R is decomposed into R1(ABC) and R2(AD) which is dependency preserving because FD A→BC is a part of relation R1(ABC).

Types of Decomposition

Decomposition of a relation can be completed in the following two ways-



Lossless Join Decomposition-

Consider there is a relation R which is decomposed into sub relations $R_1, R_2, \dots, R_n$.

This decomposition is called lossless join decomposition when the join of the sub relations results in the same relation R that was decomposed.

For lossless join decomposition, we always have- $R_1 \bowtie R_2 \bowtie R_3 \dots \bowtie R_n = R$

where $\bowtie$ is a natural join operator.

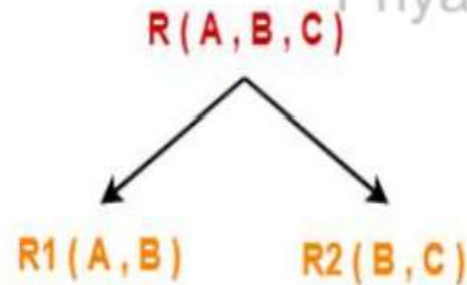
Types of Decomposition

Lossless Join Decomposition-Example-Consider the following relation $R(A, B, C)$ -Consider this relation is decomposed into two sub relations $R_1(A, B)$ and $R_2(B, C)$ -

The two sub relations are-

A	B	C
1	2	1
2	5	3
3	3	3

$R(A, B, C)$



A	B
1	2
2	5
3	3

$R_1(A, B)$

B	C
2	1
5	3
3	3

$R_2(B, C)$

Now, let us check whether this decomposition is lossless or not.

For lossless decomposition, we must have- $R_1 \bowtie R_2 = R$ Now, if we perform the natural join ($\bowtie$) of the sub relations R_1 and R_2 , we get-

Types of Decomposition

A	B	C
1	2	1
2	5	3
3	3	3

This relation is same as the original relation R.

Thus, we conclude that the above decomposition is lossless join decomposition.

NOTE-

- Lossless join decomposition is also known as non-additive join decomposition.
- This is because the resultant relation after joining the sub relations is same as the decomposed relation.
- No extraneous tuples appear after joining of the sub-relations.

Types of Decomposition

Lossy Join Decomposition- Consider there is a relation R which is decomposed into sub relations R1 , R2 , , Rn. This decomposition is called lossy join decomposition when the join of the sub relations does not result in the same relation R that was decomposed. The natural join of the sub relations is always found to have some extraneous tuples.

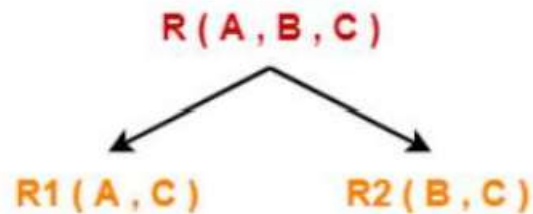
For lossy join decomposition, we always have-

$R1 \bowtie R2 \bowtie R3 \dots \bowtie Rn \supset R$ where $\bowtie$ is a natural join operator

Example- Consider the following relation R(A , B , C)-Consider this relation is decomposed into two sub relations as R1(A , C) and R2(B , C)-

A	B	C
1	2	1
2	5	3
3	3	3

R(A , B , C)



A	C
1	1
2	3
3	3

R₁(A , C)

B	C
2	1
5	3
3	3

R₂(B , C)

Types of Decomposition

Now, let us check whether this decomposition is lossy or not.

For lossy decomposition, we must have- $R1 \bowtie R2 \supset R$.

Now, if we perform the natural join ($\bowtie$) of the sub relations R1 and R2 we get-

This relation is not same as the original relation R

contains some extraneous tuples. Clearly, $R1 \bowtie R2 \supset R$.

Thus, we conclude that the above decomposition is lossy join decomposition.

NOTE-

- Lossy join decomposition is also known as careless decomposition.
- This is because extraneous tuples get introduced in the natural join of the sub-relations.
- Extraneous tuples make the identification of the original tuples difficult.

A	B	C
1	2	1
2	5	3
2	3	3
3	5	3
3	3	3

Determining Whether Decomposition Is Lossless Or Lossy-

Consider a relation R is decomposed into two sub relations R1 and R2. Then,

- If all the following conditions satisfy, then the decomposition is lossless.
- If any of these conditions fail, then the decomposition is lossy.

Condition-01: Union of both the sub relations must contain all the attributes that are present in the original relation R. Thus, $R1 \cup R2 = R$

Priyanka Dhanraj

Condition-02: Intersection of both the sub relations must not be null. In other words, there must be some common attribute which is present in both the sub relations. Thus, $R1 \cap R2 \neq \emptyset$

Condition-03: Intersection of both the sub relations must be a super key of either R1 or R2 or both.

Thus, $R1 \cap R2 = \text{Super key of R1 or R2}$

Determining Whether Decomposition Is Lossless Or Lossy-

Supplier relation :

S#	CITY	Status
S3	Mumbai	30
S5	Delhi	30

a) Lossy decomposition

S#	Status
S3	30
S5	30

CITY	Status
Mumbai	30
Delhi	30

b) Lossless decomposition

S#	Status
S3	30
S5	30

S#	CITY
S3	Mumbai
S5	Delhi

Determining Whether Decomposition Is Lossless Or Lossy-

Given R a relation and F a set of FDs

Decompose R into R1 and R2

Decomposition is lossless if F^+ contains

either $\text{Intersection}(R1, R2) \rightarrow R1$ or $\text{Intersection}(R1, R2) \rightarrow R2$

EmpDept (empno, empname, job, deptno, dname, dloc)

$F = \{ \text{deptno} \rightarrow \text{dname} \quad \text{deptno} \rightarrow \text{dloc} \quad \text{empno} \rightarrow \text{empname} \\ \text{empno} \rightarrow \text{deptno} \quad \text{empno} \rightarrow \text{job} \}$

Decompose EmpDept into two relations

Emp (empno, empname, job, deptno)

Dept(deptno, dname, dloc)

Lossless

$\text{Intersection}(\text{Emp}, \text{Dept}) = \{ \text{deptno} \} \rightarrow \text{Dept}$

Determining Whether Decomposition Is Lossless Or Lossy-

Problem-01: Consider a relation schema $R (A , B , C , D)$ with the functional dependencies $A \rightarrow B$ and $C \rightarrow D$. Determine whether the decomposition of R into $R1 (A , B)$ and $R2 (C , D)$ is lossless or lossy.

Solution- To determine whether the decomposition is lossless or lossy, We will check all the conditions one by one. If any of the conditions fail, then the decomposition is lossy otherwise lossless.

Condition-01: Union of both the sub relations must contain all the attributes of relation R . So, we have- $R1 (A , B) \cup R2 (C , D) = R (A , B , C , D)$

Clearly, union of the sub relations contain all the attributes of relation R . Thus, condition-01 satisfies.

Condition-02: Intersection of both the sub relations must not be null. So, we have-

$R1 (A , B) \cap R2 (C , D) = \Phi$ Clearly, intersection of the sub relations is null. So, condition-02 fails.

Thus, we conclude that the decomposition is lossy.

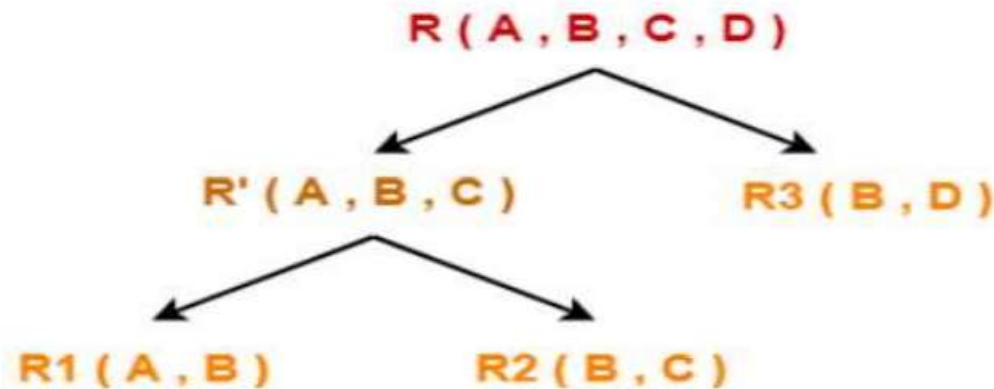
Determining Whether Decomposition Is Lossless Or Lossy-

Problem-02: Consider a relation schema $R(A, B, C, D)$ with the following functional dependencies-

$A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow B$

Determine whether the decomposition of R into $R_1(A, B)$, $R_2(B, C)$ and $R_3(B, D)$ is lossless or lossy.

Solution- Consider the original relation R was decomposed into the given sub relations as shown-



Determining Whether Decomposition Is Lossless Or Lossy-

Decomposition of $R(A, B, C, D)$ into $R'(A, B, C)$ and $R_3(B, D)$ - To determine whether the decomposition is lossless or lossy, We will check all the conditions one by one.

If any of the conditions fail, then the decomposition is lossy otherwise lossless.

Condition-01: Union of both the sub relations must contain all the attributes of relation R .

$R'(A, B, C) \cup R_3(B, D) = R(A, B, C, D)$ Thus, condition-01 satisfies.

Condition-02: Intersection of both the sub relations must not be null.

$R'(A, B, C) \cap R_3(B, D) = B$ Thus, condition-02 satisfies.

Condition-03: Intersection of both the sub relations must be the super key of one of the two sub relations or both. $R'(A, B, C) \cap R_3(B, D) = B$

Now, the closure of attribute B is- $B^+ = \{ B, C, D \}$ Now, we see- Attribute ' B ' can not determine attribute ' A ' of sub relation R' . Thus, it is not a super key of the sub relation R' . Attribute ' B ' can determine all the attributes of sub relation R_3 . Thus, it is a super key of the sub relation R_3 .

Determining Whether Decomposition Is Lossless Or Lossy-

Clearly, intersection of the sub relations is a super key of one of the sub relations. So, condition-03 satisfies. Thus, we conclude that the decomposition is lossless.

Decomposition of $R'(A, B, C)$ into $R_1(A, B)$ and $R_2(B, C)$ -

Condition-01: $R_1(A, B) \cup R_2(B, C) = R'(A, B, C)$ Thus, condition-01 satisfies.

Condition-02: $R_1(A, B) \cap R_2(B, C) = B$ Thus, condition-02 satisfies.

Condition-03: $R_1(A, B) \cap R_2(B, C) = B$ Now, the closure of attribute B is- $B^+ = \{B, C, D\}$

Clearly, intersection of the sub relations is a super key of one of the sub relations.

So, condition-03 satisfies. Thus, we conclude that the decomposition is lossless.

Conclusion-

Overall decomposition of relation R into sub relations R_1 , R_2 and R_3 is lossless.

Dependency Preservation

Dependency Preservation- Dependency preservation ensures-

- None of the functional dependencies that holds on the original relation are lost.

A Decomposition $D = \{ R_1, R_2, R_3, \dots, R_n \}$ of R is dependency preserving wrt a set F of Functional dependency if $(F_1 \cup F_2 \cup \dots \cup F_m)^+ = F^+$.

Consider a relation $R : R \rightarrow F \{ \dots \text{with some functional dependency (FD)} \dots \}$

R is decomposed or divided into R_1 with FD $\{ f_1 \}$ and R_2 with $\{ f_2 \}$, then there can be three cases:

$f_1 \cup f_2 = F \rightarrow$ Decomposition is dependency preserving.

$f_1 \cup f_2$ is a subset of $F \rightarrow$ Not Dependency preserving.

$f_1 \cup f_2$ is a super set of $F \rightarrow$ This case is not possible.

Dependency Preservation

EmpDept (empno, empname, job, deptno, dname, dloc)

$F = \{ \text{deptno} \rightarrow \text{dname} \quad \text{deptno} \rightarrow \text{dloc} \quad \text{empno} \rightarrow \text{empname}$
 $\quad \text{empno} \rightarrow \text{deptno} \quad \text{empno} \rightarrow \text{job} \}$

Decompose EmpDept into two relations

Emp (empno, empname, job, deptno)

$F_{\text{emp}} = \{ \text{empno} \rightarrow \text{empname}, \quad \text{empno} \rightarrow \text{deptno},$
 $\quad \text{empno} \rightarrow \text{job} \}$

Dept(deptno, dname, dloc)

$F_{\text{dept}} = \{ \text{deptno} \rightarrow \text{dname}, \quad \text{deptno} \rightarrow \text{dloc} \}$

$F^| = F_{\text{emp}} \cup F_{\text{dept}} = F$ hence dependency preserving

Dependency Preservation

Problem: Let a relation $R(A, B, C, D)$ and functional dependency $\{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Relation R is decomposed into $R_1(A, B, C)$ and $R_2(C, D)$. Check whether decomposition is dependency preserving or not.

Solution: $R_1(A, B, C)$ and $R_2(C, D)$ Let us find closure of F_1 and F_2

To find closure of F_1 , consider all combination of ABC . i.e., find closure of A, B, C, AB, BC and AC

Note ABC is not considered as it is always ABC

$\text{closure}(A) = \{A\}$ // Trivial

$\text{closure}(B) = \{B\}$ // Trivial

$\text{closure}(C) = \{C, A, D\}$ but D can't be in closure as D is not present R_1 . $= \{C, A\}$

$C \rightarrow A$ // Removing C from right side as it is trivial attribute

$\text{closure}(AB) = \{A, B, C, D\} = \{A, B, C\}$

$AB \rightarrow C$ // Removing AB from right side as these are trivial attributes

Dependency Preservation

$\text{closure}(BC) = \{B, C, D, A\} = \{A, B, C\}$

$BC \twoheadrightarrow A$ // Removing BC from right side as these are trivial attributes

$\text{closure}(AC) = \{A, C, D\}$ but D can't be in closure as D is not present R1

// Removing AC from right side as these are trivial attributes

$F1 \{C \twoheadrightarrow A, AB \twoheadrightarrow C, BC \twoheadrightarrow A\}$. Similarly $F2 \{C \twoheadrightarrow D\}$

In the original Relation Dependency $\{AB \twoheadrightarrow C, C \twoheadrightarrow D, D \twoheadrightarrow A\}$.

$AB \twoheadrightarrow C$ is present in $F1$.

$C \twoheadrightarrow D$ is present in $F2$.

$D \twoheadrightarrow A$ is not preserved.

$F1 \cup F2$ is a subset of F . So given decomposition is not dependency preserving.

Exercise

Given $R(A, B, C, D)$ with A as primary key, and

$A \rightarrow B$

$A \rightarrow C$

$B \rightarrow D$

Determine which are 'good' decompositions

$R_1(A, B, C)$ and $R_2(B, D)$

Bad: Lossy, FD preserving

$R_1(A, B, D)$ and $R_2(B, C)$

Bad: Lossy, FD non-preserving

$R_1(A, B, D)$ and $R_2(A, C)$

Good: Non-lossy, FD preserving

Normalization

A large database defined as a single relation may result in data duplication. **Data redundancy** occurs in a relational database when two or more rows or columns have the same value or repetitive value. This repetition of data may result in:

- Making relations very large.
- It isn't easy to maintain and update data as it would involve searching many records in relation.
- Wastage and poor utilization of disk space and resources.
- The likelihood of errors and inconsistencies increases.

So to handle these problems, we should analyze and decompose the relations with redundant data into smaller, simpler, and well-structured relations that satisfy desirable properties. Normalization is a process of decomposing the relations into relations with fewer attributes. Normalization divides the larger table into smaller and links them using relationships.

Normalization : Definition

Normalization is done through normal forms. The technique eliminates the following: Multivalued attributes , Doubled or **repeated attributes**, **Non-descriptive** attributes ,Composite attributes, Attributes with **redundant information**.

It provides a well-functioning information environment. The method systematically ensures:

Priyanka Dhanraj

- A database structure **suitable for generalized queries**.
- **Minimized data redundancy increasing memory efficiency on a database server**.
- **Maximized data integrity** through the reduced insert, update, and delete anomalies (Redundancy due to inconsistency is DB anomalies).

Database normalization transforms overall database consistency, providing an efficient environment.

Need of Normalization

The **main reason** for normalizing the relations is **removing these anomalies**. Failure to eliminate anomalies leads to data redundancy (Left table show redundancy) and can cause data integrity and other problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure. (Right table for anomalies example)

Relation R (As previous table)

Sid	Sname	Cid	Cname	Fid	Fname	Sal
S1	A	C1	C	F1	X	50K
S2	A	C1	C	F1	X	50K
S3	B	C1	C	F1	X	50K
S4	B	C2	C++	F2	Y	40K
S5	B	C1	C	F1	X	50K

Priyanka Dhanraj

Emp_Id	Emp_Name	Emp_Address	Emp_Dept
101	Rick	Delhi	D001
101	Rick	Delhi	D002
123	Maggie	Agra	D890
166	Glenn	Chennai	D900
166	Glenn	Chennai	D004

Need of Normalization

Update anomaly: In the above table we have two rows for employee Rick as he belongs to two departments of the company. If we want to update the address of Rick then we have to update the same in two rows or the data will become inconsistent. If somehow, the correct address gets updated in one department but not in other then as per the database, Rick would be having two different addresses, which is not correct and would lead to inconsistent data.

Insert anomaly: Suppose a new employee joins the company, who is under training and currently not assigned to any department then we would not be able to insert the data into the table if Emp_Dept field doesn't allow null.

Delete anomaly: Let's say in future, company closes the department D890 then deleting the rows that are having Emp_Dept as D890 would also delete the information of employee Maggie since she is assigned only to this department.

Types of Normal Forms

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Following are the various types of Normal forms:

	1NF	2NF	3NF	4NF	5NF
Decomposition of Relation	R	R ₁₁ R ₁₂	R ₂₁ R ₂₂ R ₂₃	R ₃₁ R ₃₂ R ₃₃ R ₃₄	R ₄₁ R ₄₂ R ₄₃ R ₄₄ R ₄₅
Conditions	Eliminate Repeating Groups	Eliminate Partial Functional Dependency	Eliminate Transitive Dependency	Eliminate Multi-values Dependency	Eliminate Join Dependency

Types of Normal Forms

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key. Priyanka Dhanraj
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages and Disadvantages of Normalization

Advantages of Normalization

1. Normalization helps to minimize data redundancy.
2. Greater overall database organization.
3. Data consistency within the database.
4. Much more flexible database design. Priyanka Dhanraj
5. Enforces the concept of relational integrity.

Disadvantages of Normalization

1. You cannot start building the database before knowing what the user needs.
2. The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
3. It is very time-consuming and difficult to normalize relations of a higher degree.
4. Careless decomposition may lead to a bad database design, leading to serious problems.

History of Normalization

- The concept of normalization was first proposed by Edgar F. Codd in 1970, when he proposed the first normal form (1NF) in his paper A Relational Model of Data for Large Shared Data Banks (this is the paper in which he introduced the whole idea of relational databases).
- Codd continued his work on normalization and defined the second normal form (2NF) and third normal form (3NF) in 1971.
- Codd then teamed up with Raymond F. Boyce to define the Boyce-Codd normal form (BCNF) in 1974.
- Ronald Fagin introduced the fourth normal form (4NF) in 1977.
- Fagin then introduced the fifth normal form (5NF) in 1979.
- Fagin then introduced the domain key normal form (DKNF) in 1981.
- Carlo Zaniolo introduced the elementary key normal form (EKNF) in 1982.
- Ronald Fagin then teamed up with Hugh Darwen and C.J. Date to produce the essential tuple normal form (ETNF) in 2012.

Normalization in DBMS

The elementary concepts used in database normalization are:

1. **Keys** : Column attributes that identify a database record uniquely. The types of keys used in normalization are:

Super Key : A set of features that uniquely define each record in a table.

Candidate Key : Keys selected from the set of super keys where the number of fields is minimal.

Primary Key : The most appropriate choice from the set of candidate keys serves as the table's primary key.

Foreign Key : The primary key of another table.

Composite Key : Two or more attributes together form a unique key but are not keys individually.

As tables decompose into multiple simpler tables, **keys define a point of reference for a database entity.**

Normalization in DBMS

2. Functional Dependencies : Constraints between two attributes in a relation. In the long run, the dependencies help determine the overall quality of a database.

3. Normal Forms : Steps to accomplish a certain quality of a database. Normalizing a database is achieved through a set of rules known as normal forms. All levels of normalization are cumulative. Previous normal form requirements must be satisfied before moving on to the following form.



Normalization in DBMS

A database is normalized when it fulfills the third normal form. Further steps in normalization make the database design complicated and could compromise the functionality of the system.

The stages of normalization are:

Stage	Redundancy Anomalies Addressed
Unnormalized Form (UNF)	The state before any normalization. Redundant and complex values are present.
First Normal Form (1NF)	Repeating and complex values split up, making all instances atomic.
Second Normal Form (2NF)	Partial dependencies decompose to new tables. All rows functionally depend on the primary key.
Third Normal Form (3NF)	Transitive dependencies decompose to new tables. Non-key attributes depend on the primary key.
Boyce-Codd Normal Form (BCNF)	Transitive and partial functional dependencies for all candidate keys decompose to new tables.
Fourth Normal Form (4NF)	Removal of multivalued dependencies.
Fifth Normal Form (5NF)	Removal of JOIN dependencies.

Normalization in DBMS : First Normal Form

First Normal Form- A given relation is called in First Normal Form (1NF) if each cell of the table contains only an atomic value. OR

A given relation is called in First Normal Form (1NF) if the attribute of every tuple is either single valued or a null value.

Example- The following relation is not in 1NF- However, This relation can be brought into 1NF.

This can be done by rewriting the relation such that each cell of the table contains only one value.

Student_id	Name	Subjects
100	Akshay	Computer Networks, Designing
101	Aman	Database Management System
102	Anjali	Automata, Compiler Design

Relation is not in 1NF

Normalization in DBMS : First Normal Form

This relation is in First Normal Form (1NF).

NOTE- By default, every relation is in 1NF. This is because formal definition of a relation states that value of all the attributes must be atomic.

Student_id	Name	Subjects
100	Akshay	Computer Networks
100	Akshay	Designing
101	Aman	Database Management System
102	Anjali	Automata
102	Anjali	Compiler Design

Relation is in 1NF

Normalization in DBMS : First Normal Form

- The table cells must be of a single value.
- Eliminate repeating groups in individual tables (In relational terms, a table is in the first normal form if it contains no repeating columns.).
- Create a separate table for each set of related data.
- Identify each set of related data with a primary key.

IMP: In 1NF relation, the order of tuples (rows) and attributes (columns) does not matter.

Example

order number	customer name	contact person	total amount
1	Rishabh	Manish	134.23
2	Preeti	Rohan	521.24
3	Rishabh	Manish	1042.42
4	Rishabh	Manish	928.53

- The above relation satisfies the properties of relation and is said to be in first normal form (or 1NF). Conceptually it is convenient to have all the information in one relation since it is then likely to be easier to query the database.

Normalization in DBMS : First Normal Form (Example)

Process to convert in 1NF : Which attribute creates a problem (Because of multi-valued) separate it into with the primary key of the main table. And keep the rest of the attributes with the primary key in the another table.

Roll no	Name	Branch	Class 1	Class 2
1	AAA	CSE	102	104
2	BBB	ECE	209	102

Convert into 1NF

Roll_no	Name	Branch	Class #
1	AAA	CSE	102
1	AAA	CSE	104
2	BBB	ECE	209
2	BBB	ECE	102

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1

Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

Normalization in DBMS : First Normal Form

Example :

- STUDENT(name, fname, roll-no, course, grade)

Every attribute takes on a simple value. Thus it is in 1 NF.

- EMPLOYEE(name, address, child)

child has attributes like child-name, age, sex. It is not atomic and thus is not in 1 NF.

- PRODUCT(product-no, price, qty)

It is in 1 NF as every attribute has an atomic value

Normalization in DBMS : First Normal Form

ENFORCING THE 1 NF

- Replacement method

Systematically replaces all complex attributes by their constituents

Example: For EMPLOYEE (name, address, child) define as

EMPLOYEE(name, address, child-name, cage, csex)

- Decomposition method

Split the relation into two components, each of which are in 1NF.

Example: For EMPLOYEE define

EMPLOYEE(ename, address) and CHILD(cname, ename, cage, asex)

Normalization in DBMS : Second Normal Form

- **Definition:** A relation is in 2NF if it is in 1NF and every non-key attribute is fully dependent on each candidate key of the relation (Remove Partial Dependencies).
- Functional Dependency : The value of one attribute in a table is determined entirely by the value of another.
- Partial Dependency: A type of functional dependency where an attribute is functionally dependent on only part of the primary key (primary key must be a composite key).

A given relation is called in Second Normal Form (2NF) if and only if-

- Relation already exists in 1NF and No partial dependency exists in the relation.

In other words, $A \rightarrow B$ is called a partial dependency if and only if-

- A is a subset of some candidate key , B is a non-prime attribute.
- If any one condition fails, then it will not be a partial dependency.

Note : Prime attribute is the attribute which is the part of the candidate key. Non -prime attribute is the attribute which is not the part of the candidate key.

Normalization in DBMS : Second Normal Form

Fully functionally dependent : In the relation $ABC \rightarrow D$, attribute D is fully functionally dependent on ABC and not on any proper subset of ABC. That means that subsets of ABC like AB, BC, A, B, etc cannot determine D.

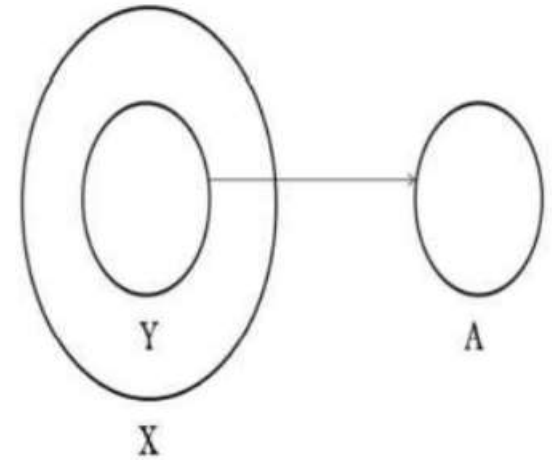
Partially dependency : If the proper subset of candidate key determines non-prime attribute, it is called partial dependency. For example, we have a relationship $AC \rightarrow B$, $A \rightarrow D$, and $D \rightarrow B$.

Now if we compute the closure of $\{A^+\} = ADB$ Here A is alone capable of determining B, which means B is partially dependent on AC.

NOTE-

- To avoid partial dependency, incomplete candidate key must not determine any non-prime attribute.
- However, incomplete candidate key can determine prime attributes.

Partial Dependency (PD):



Normalization in DBMS : Second Normal Form (Example)

- **Definition:** A relation is in 2NF if it is in 1NF and every non-key attribute is fully dependent on each candidate key of the relation. **Example :** The following relation is not in Second Normal Form.

Order number	Customer name	Contact person	Total amount
1	Rishabh	Manish	134.23
2	Preeti	Rohan	521.24
3	Rishabh	Manish	1042.42
4	Rishabh	Manish	928.53

In the table above, the order number serves as the primary key. Notice that the customer and total amount are dependent upon the order number -- this data is specific to each order. However, the contact person is dependent upon the customer. An alternative way to accomplish this would be to create two tables:

Customer name	Contact person
Rishabh	Manish
Preeti	Rohan

order number	Customer name	Total amount
1	Rishabh	134.23
2	Preeti	521.24
3	Rishabh	1042.42
4	Rishabh	928.53

Normalization in DBMS : Second Normal Form (Example)

- Note that if the primary key is not a composite key, all non-key attributes are always fully functional dependent on the primary key.
- Consider the following example: This TABLE_PURCHASE_DETAIL (LEFT) has a composite primary key [Customer ID, Store ID]. The non-key attribute is [Purchase Location]. In this case, [Purchase Location] only depends on [Store ID], which is only part of the primary key. Therefore, this table does not satisfy second normal form.
- What we have done is to remove the partial functional dependency that we initially had. Now, in the table [TABLE_STORE], the column [Purchase Location] is fully dependent on the primary key of that table, which is [Store ID].

TABLE_PURCHASE_DETAIL

Customer ID	Store ID	Purchase Location
1	1	Los Angeles
1	3	San Francisco
2	1	Los Angeles
3	2	New York
4	3	San Francisco

TABLE_PURCHASE

Customer ID	Store ID
1	1
1	3
2	1
3	2
4	3

TABLE_STORE

Store ID	Purchase Location
1	Los Angeles
2	New York
3	San Francisco

Normalization in DBMS : Second Normal Form (Example)

Anomalies Due to PFD

Consider a relation **Supplier(S, P#, CITY)**

S	P#	CITY
X	1	DELHI
X	2	DELHI
X	3	DELHI
Y	1	MUMBAI
Y	2	MUMBAI

Let the dependencies be

S, P# $\longrightarrow$ CITY

S $\longrightarrow$ CITY

Normalization in DBMS : Second Normal Form (Example)

Supplier relation can be split into two components as

S1(S, P#) key S,P#

and

S2(S, CITY) key S

S	P#
X	1
X	2
X	3
Y	1
Y	2

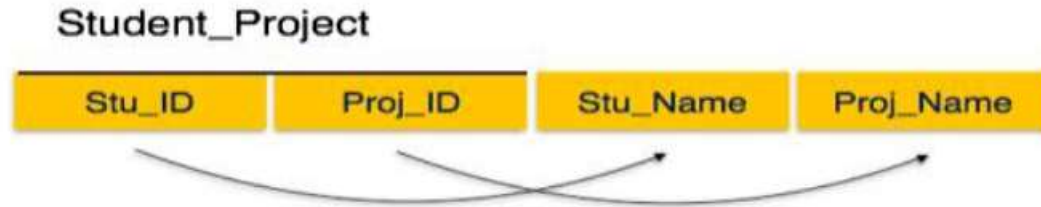
S	City
X	DELHI
Y	MUMBAI

Priyanka Dhanraj

- The fact that S operates from a CITY is represented only once.
- When operating on S2 there is no interference from S1.
- When operating on S1 there is no interference from S2.

Normalization in DBMS : Second Normal Form (Example)

- Example : In 2NF (Right)



Student

Stu_ID	Stu_Name	Proj_ID
--------	----------	---------

Project

Proj_ID	Proj_Name
---------	-----------

- Example :

(1st table is not 2NF)

Example:

Let us consider following table which is in first normal form:

Employee No	Department No	Employee Name	Department
-------------	---------------	---------------	------------

Make 2NF

(Last two

Table)

Table 1:Employee_NO table

Employee No	Department No	Employee Name
-------------	---------------	---------------

Table 2:Department table

Department No	Department
---------------	------------

Normalization in DBMS : Second Normal Form

- Example- Consider a relation- $R (V , W , X , Y , Z)$ with functional dependencies-

$$VW \rightarrow XY , Y \rightarrow V , WX \rightarrow YZ$$

The possible candidate keys for this relation are- VW , WX , WY

From here, Prime attributes = $\{ V , W , X , Y \}$, Non-prime attributes = $\{ Z \}$

Now, if we observe the given dependencies-

- There is no partial dependency.
- This is because there exists no dependency where incomplete candidate key determines any non-prime attribute.
- Thus, we conclude that the given relation is in 2NF.

Normalization in DBMS : Second Normal Form

Example:

R (ABCD)

Candidate Key (CK) – AB

Prime Attribute (PA) = {A, B}

Non – Prime Attribute (NPA) = {C, D}

FDs = {A → CD, B → D, B → AC }.

Priyanka Dhanraj

- Example: Then $A \rightarrow CD$ is a Partial Dependency (PD) because $A \rightarrow AB$ and 'CD' is Non-Prime Attribute (NPA), same $B \rightarrow D$ is Partial dependency because $B \subset AB$ and D is NPA.
- What about $B \rightarrow AC$ [where A – Prime Attribute and C – Non-Prime Attribute] then we decompose it $B \rightarrow A$ (FD), $B \rightarrow C$ (PD).
- So, $B \rightarrow AC$ not 2NF because partial dependency exists $B \rightarrow C$.

Normalization in DBMS : Second Normal Form

- Let R (ABCD) and CK = {B, A, E} in this case no Partial Dependency is possible because all Candidate Key (CK) is single not compound. So, if it is single then subset is not possible ($Y \subset CK$). So, Partial Dependency is not possible.
- So if all CK of a relation is atomic (single) or all attribute of a relation prime attribute then this relation is in 2NF i.e. no Partial Dependency is possible.

Priyanka Dhanraj

Normalization in DBMS : Third Normal Form

In 3rd NF

- Remove transitive dependencies.
- Transitive Dependency : A type of functional dependency where an attribute is functionally dependent on an attribute other than the primary key. Thus its value is only indirectly determined by the primary key.
- Create a separate table containing the attribute and the fields that are functionally dependent on it. The tables created at this step will usually contain descriptions of either resources or agents. Keep a copy of the key attribute in the original file.

Priyanka Dhanraj

Normalization in DBMS : Third Normal Form

Example

Company	City	State	ZIP
ABC Ltd.	Mumbai	MH	10169
XYZ Ltd.	Noida	UP	33196
ASD Ltd.	Chennai	TN	21046

The above table is not in the 3NF.

In this example, the city and state are dependent upon the ZIP code. To place this table in 3NF, two separate tables would be created -- one containing the company name and ZIP code and the other containing city, state, ZIP code pairings.

Company	ZIP
ABC Ltd.	10169
XYZ Ltd.	33196
ASD Ltd.	21046

City	State	ZIP
Mumbai	MH	10169
Noida	UP	33196
Chennai	TN	21046

This may seem overly complex for daily applications and indeed it may be. Database designers should always keep in mind the tradeoffs between higher level normal forms and the resource issues that complexity creates.

Normalization in DBMS : Third Normal Form

3rd Normal Form Example:

In the table, [Book ID] determines [Genre ID], and [Genre ID] determines [Genre Type]. Therefore, [Book ID] determines [Genre Type] via [Genre ID] and we have transitive functional dependency, and this structure does not satisfy third normal form. To bring this table to third normal form, we split the table into two as follows:

TABLE_BOOK_DETAIL

Book ID	Genre ID	Genre Type	Price
1	1	Gardening	25.99
2	2	Sports	14.99
3	1	Gardening	10.00
4	3	Travel	12.99
5	2	Sports	17.99

Now all non-key attributes are fully functional dependent only on the primary key. In [TABLE_BOOK], both [Genre ID] and [Price] are only dependent on [Book ID]. In [TABLE_GENRE], [Genre Type] is only dependent on [Genre ID].

TABLE_BOOK

Book ID	Genre ID	Price
1	1	25.99
2	2	14.99
3	1	10.00
4	3	12.99
5	2	17.99

TABLE_GENRE

Genre ID	Genre Type
1	Gardening
2	Sports
3	Travel

Normalization in DBMS : Third Normal Form

A given relation is called in Third Normal Form (3NF) if and only if-

1. Relation already exists in 2NF.
2. No transitive dependency exists for non-prime attributes.

Transitive Dependency : $A \rightarrow B$ is called a transitive dependency if and only if-

1. A is not a super key.
2. B is a non-prime attribute (B is not part of some candidate key).

If any one condition fails, then it is not a transitive dependency.

NOTE-

- Transitive dependency must not exist for non-prime attributes.
- However, transitive dependency can exist for prime attributes.

Normalization in DBMS : Third Normal Form

A relation is called in Third Normal Form (3NF) if and only if- **Any one condition holds** for each non-trivial functional dependency $A \rightarrow B$: A is a super key , B is a prime attribute

Example:

Let **R (ABCDEF)**, **CK = {AB, CD}**, **PA = {A, B, C, D}**, **NPA = {E, F}**

$ABE \rightarrow F$ is in **3NF** because '**ABE**' is **Super Key**.

$A \rightarrow E$ is not in **3NF** because '**A**' is **not Super Key** and '**E**' is **NPA**.

$A \rightarrow BE$ then checks **$A \rightarrow B$** , **$A \rightarrow E$** .

$A \rightarrow B$ is in **3NF** because '**A**' is **not Super Key** but '**B**' is **PA**.

$A \rightarrow E$ is **not in 3NF** because '**A**' is **not Super Key** and '**E**' is **NPA**.

So, because **$A \rightarrow E$** is **Transitive Dependency** then **$A \rightarrow BE$** is **not in 3NF**.

Normalization in DBMS : Third Normal Form

Example- Consider a relation- $R (A , B , C , D , E)$ with functional dependencies-

$A \rightarrow BC$, $CD \rightarrow E$, $B \rightarrow D$, $E \rightarrow A$

The possible candidate keys for this relation are- A , E , CD , BC

From here, Prime attributes = $\{ A , B , C , D , E \}$ and There are no non-prime attributes

Now, It is clear that there are no non-prime attributes in the relation.

In other words, all the attributes of relation are prime attributes.

Thus, all the attributes on RHS of each functional dependency are prime attributes.

Thus, we conclude that the given relation is in 3NF.

Normalization in DBMS : Second Normal Form

Another Example:

Let **R (ABCDE)**,

$$\mathbf{F = \{AB \rightarrow C, C \rightarrow D, B \rightarrow E\} (AB)^+ = ABCDE}$$

So, **AB** is **CK**.

$$\mathbf{CK = \{AB\}, PA = \{A, B\}, NPA = \{C, D, E\}}$$

Now **F = {AB → C, C → D, B → E}** check which **FD** is in **2NF**.

Let **AB → C** is in **2NF** because **AB** is not a proper subset of **CK** (**AB** itself **CK**)

C → D is in **2NF** because of **C ∉ CK**.

B → E is **not in 2NF** because **B ⊂ AB** and **E** are **NPA**.

So, here **B → E** creates **redundancy**.

Normalization in DBMS : Second Normal Form

- Primary key of R, here only candidate key is AB. So, because we know $B \rightarrow E$ creates redundancy, then we know the point from where we decompose the table.



- One attribute should be common in both tables to keep lossless decomposition and dependency preserving. How redundancy removes which only created by $B \rightarrow E$. May be R1 contain some redundancy but here we only remove redundant which is from $B \rightarrow E$.
- Now there is no Partial Dependency here because Candidate Key of R1 is AB and R2 is B.

So, $AB \rightarrow C$ (FD), $C \rightarrow D$ (FD) and at R_2 $B \rightarrow E$ (FD).

Now R_1 and R_2 is in **2NF**.

Normalization in DBMS : Third Normal Form

$R_1(ABCD)$:

$AB \rightarrow C, C \rightarrow D$

$CK(AB), PA = \{A, B\}, NPA = \{C, D\}$

Here **$AB \rightarrow C$ – Functional Dependency** because **AB is Super Key** but **$C \rightarrow D$ Transitive Dependency** because **C is not Super Key**, and **D is Non-Prime Attribute**.

So, **R_1 is not in 3NF. 2NF but not in 3NF.**

$R_2 (BE)$:

$B \rightarrow E$

$CK(B), PA = \{B\}, NPA = \{E\}$

Here **$B \rightarrow E$ – Functional Dependency** because **B is Super Key**.

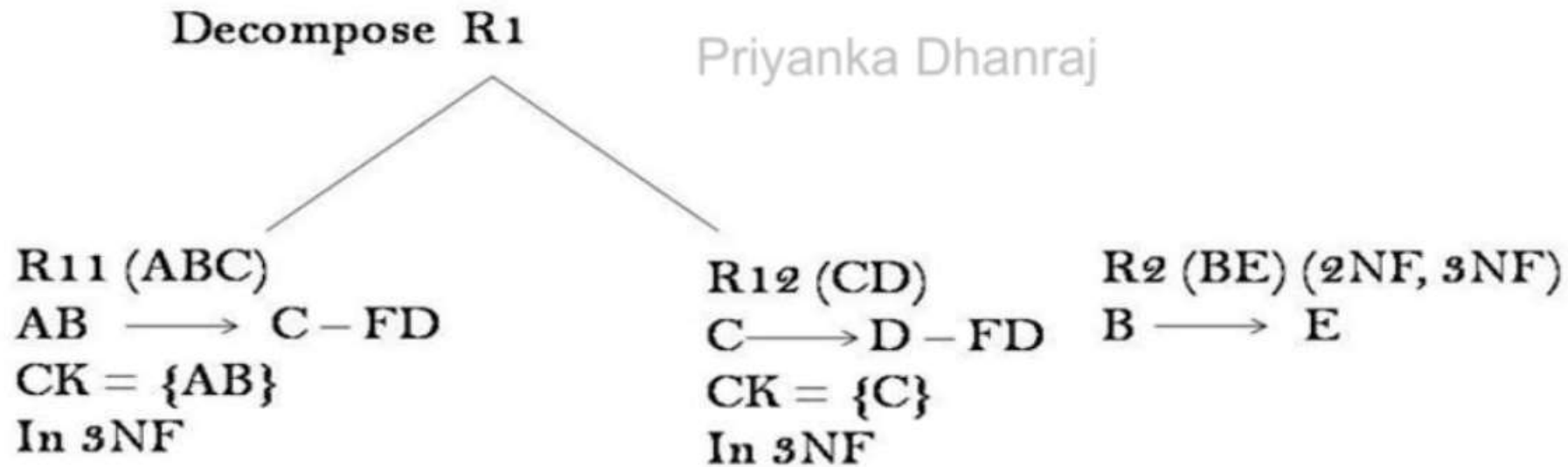
So, **R_2 is in 3NF.**

Finally **R_1 is in 2NF and R_2 is in 3NF.**

Normalization in DBMS : Third Normal Form

So, this **redundancy** is coming from **Transitive Dependency** $C \rightarrow D$.

So, we can **decompose** '**CD**' which creates the problem and keep the rest of the attribute on one side.



Priyanka Dhanraj

Normalization in DBMS : BCNF

BCNF (Boyce Codd Normal Form) is the advanced version of 3NF. A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table. For BCNF, the table should be in 3NF, and for every FD, LHS is super key.

Example

Consider a relation R with attributes (student, subject, teacher).

Student	Teacher	Subject
Jhansi	P.Naresh	Database
jhansi	K.Das	C
subbu	P.Naresh	Database
subbu	R.Prasad	C

F: { (student, Teacher) $\rightarrow$ subject
(student, subject) $\rightarrow$ Teacher
Teacher $\rightarrow$ subject }

Normalization in DBMS : BCNF

Candidate keys are (student, teacher) and (student, subject).

The above relation is in 3NF [since there is no transitive dependency]. A relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$, X must be a key.

The above relation is not in BCNF, because in the FD (teacher $\rightarrow$ subject), teacher is not a key. This relation suffers with anomalies – For example, if we try to delete the student Subbu, we will lose the information that R. Prasad teaches C. These difficulties are caused by the fact the teacher is determinant but not a candidate key.

Decomposition for BCNF

Teacher $\rightarrow$ subject violates BCNF [since teacher is not a candidate key].

If $X \rightarrow Y$ violates BCNF then divide R into $R_1(X, Y)$ and $R_2(R-Y)$.

So R is divided into two relations $R_1(\text{Teacher, subject})$ and $R_2(\text{student, Teacher})$. All the anomalies which were present in R, now removed in the above two relations.

R1

Teacher	Subject
P.Naresh	database
K.DAS	C
R.Prasad	C

R2

Student	Teacher
Jhansi	P.Naresh
Jhansi	K.Das
Subbu	P.Naresh
Subbu	R.Prasad

Normalization in DBMS : BCNF

- A given relation is called in BCNF if and only if-

1. Relation already exists in 3NF.
2. For each non-trivial functional dependency $A \rightarrow B$, A is a super key of the relation. i.e. A cannot be a non-prime attribute when B is a prime attribute.

Sometimes BCNF is also referred as 3.5 Normal Form.

Example- Consider a relation- $R (A , B , C)$ with the functional dependencies-

$$A \rightarrow B, B \rightarrow C, C \rightarrow A$$

The possible candidate keys for this relation are- A , B , C

Now, we can observe that RHS of each given functional dependency is a candidate key.

Thus, we conclude that the given relation is in BCNF.

Note : BCNF decomposition does not always satisfy dependency preserving property. After BCNF decomposition if dependency is not preserved then we have to decide whether we want to remain in BCNF or rollback to 3NF. This process of rollback is called denormalization.

Normalization in DBMS : 4 NF

In 4th NF

- An entity is in Fourth Normal Form (4NF) when it meets the requirement of being in Third Normal Form (3NF) and additionally : Has no multiple sets of multi-valued dependencies.
- In other words, 4NF states that no entity can have more than a single one-to-many relationship within an entity if the one-to-many attributes are independent of each other.
- Definition: A table is in fourth normal form (4NF) if and only if it is in BCNF and contains no more than one multi-valued dependency.
- If no database table instance contains two or more, independent and multivalued data describing the relevant entity, then it is in 4th Normal Form.

Normalization in DBMS : 4 NF

Example

Take an example of Employee Table

info (Employee, Skills, Hobbies)

Employee	Skills	Hobbies
1	Programming	Golf
1	Programming	Bowling
1	Analysis	Golf
1	Analysis	Bowling
2	Analysis	Golf
2	Analysis	Gardening
2	Management	Golf
2	Management	Gardening

Skills (Employee, Skills)

Employee	Skills
1	Programming
1	Analysis
2	Analysis
2	Management

Hobbies (Employee, Hobby)

Employee	Hobbies
1	Golf
1	Bowling
2	Golf
2	Gardening

This table is difficult to maintain since adding a new hobby requires multiple new rows corresponding to each skill. This problem is created by the pair of multi-valued dependencies EMPLOYEE -> SKILLS and EMPLOYEE -> HOBBIES. A much better alternative would be to decompose INFO into two relations:

Normalization in DBMS : 5 NF

A Relation is considered to be in 5NF, if and only if -

- **Definition:** A table is in the fifth normal form (5NF) or Project-Join Normal Form (PJNF) if it is in 4NF and it cannot have a lossless decomposition into any number of smaller tables.
- If we can further break down the table to remove redundancy and anomaly and then re-join the decomposed tables using candidate keys, we should not lose the original data or create a new record set. In other words, combining two or more deconstructed tables should not result in the loss of records or the creation of new records. 5NF is also called the Project-join normal form (PJ/NF).
- Whenever we can recreate a table by simply joining various tables where each of these tables consists of a subset of the table's attribute, then this table is known as a Join Dependency. Thus, it is like a generalization of MVD. We can relate the JD to 5NF. Herein, a relation can be in 5NF only when it's already in the 4NF. Remember that it cannot be further decomposed.
- Let the joining of the two relations R_1 and R_2 with C be equal to the relation that is R . Then it is safe to conclude that there is a join dependency.

Normalization in DBMS : 5 NF

Example

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

Example : 1 In the above table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data.

Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

Normalization in DBMS : 5 NF

P1

SEMESTER

SUBJECT

Semester 1

Computer

Semester 1

Math

Semester 1

Chemistry

Semester 2

Math

P3

SEMSTER

LECTURER

Semester 1

Anshika

Semester 1

John

Semester 1

John

Semester 2

Akash

Semester 1

Praveen

P2

SUBJECT

LECTURER

Computer

Anshika

Computer

John

Math

John

Math

Akash

Chemistry

Praveen

Normalization in DBMS : 5 NF

- Example 2 : Take the example of a buying table. This is used to track buyers, what they buy, and from whom they buy. Take the following sample data:

Buyer	Vendor	Item
Shalley	Kashmir House	Jeans
Mary	Kashmir House	Jeans
Shalley	Radhika Sarees	Saree
Mary	Radhika Sarees	Saree
Shalley	Radhika Sarees	Suit

- The problem with the above table structure is that **if Claiborne starts to sell Jeans, then how many records must you create to record this fact?** The problem is there are pairwise cyclical dependencies in the primary key. That is, in order to determine the item, **you must know the buyer and vendor**, and to **determine the vendor you must know the buyer and the item**, and finally, **to know the buyer you must know the vendor and the item**.
- And the solution is to break this one table into three tables; Buyer-Vendor, Buyer-Item, and Vendor-Item. So the following tables are in the 5NF.

Normalization in DBMS : 5 NF

BUYER-VENDOR

Buyer	Vendor
Shalley	Kashmir House
Mary	Kashmir House
Shalley	Radhika Sarees
Mary	Radhika Sarees

BUYER-ITEM

Priyanka Dhanraj

Buyer	Item
Shalley	Jeans
Mary	Jeans
Shalley	Saree
Mary	Saree
Shalley	Suit

VENDOR- ITEM

Vendor	Item
Kashmir House	Jeans
Radhika Sarees	Saree
Radhika Sarees	Suit

Informal Design Guidelines for Relation Schemas

Relational database design: The grouping of attributes to form "good" relation schemas

- Two levels of relation schemas: The logical "user view" level, The storage "base relation" level
- Design is concerned mainly with base relations
- Criteria for "good" base relations:
 1. Discuss informal guidelines for good relational design
 2. Discuss formal concepts of functional dependencies and normal forms 3NF, BCNF

Four *informal guidelines* that may be used as *measures to determine the quality* of relation schema design:

1. Making sure that the semantics of the attributes is clear in the schema
2. Reducing the redundant information in tuples
3. Reducing the NULL values in tuples
4. Disallowing the possibility of generating spurious tuples

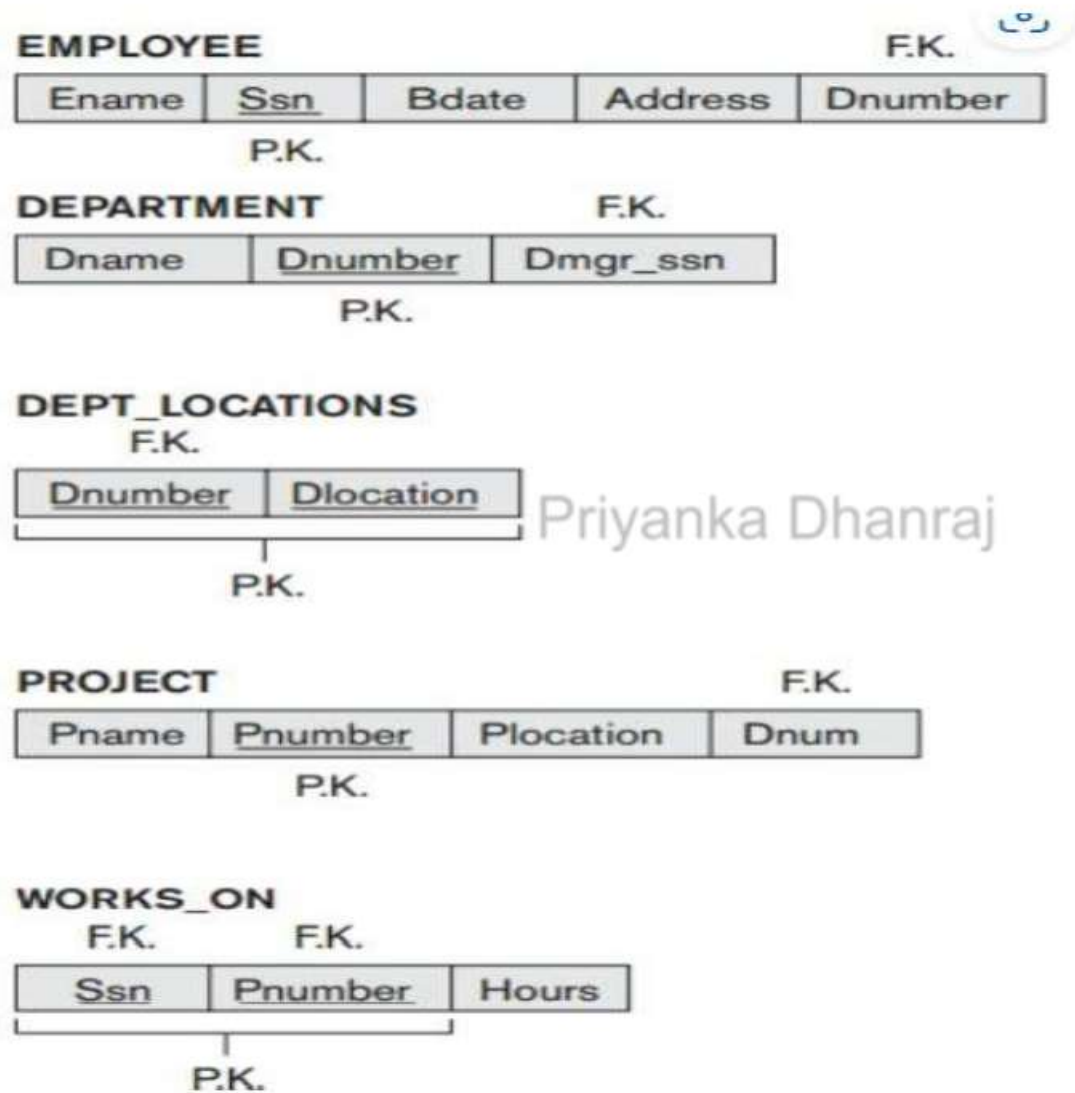
Semantics of the Relation Attributes

Each tuple in a relation should represent one entity or relationship instance

- Only foreign keys should be used to refer to other entities
- Entity and relationship attributes should be kept apart as much as possible
- Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.

Priyanka Dhanraj

Simplified version of the COMPANY relational database schema.



EMPLOYEE

ENAME	<u>SSN</u>	BDATE	ADDRESS	DNUMBER
Smith,John B.	123456789	1965-01-09	731 Fondren,Houston,TX	5
Wong,Franklin T.	333445555	1955-12-08	638 Voss,Houston,TX	5
Zelaya,Alicia J.	999887777	1968-07-19	3321 Castle,Spring,TX	4
Wallace,Jennifer S.	987654321	1941-06-20	291 Berry,Bellaire,TX	4
Narayan,Remesh K.	666884444	1962-09-15	975 Fire Oak,Humble,TX	5
English,Joyce A.	453453453	1972-07-31	5631 Rice,Houston,TX	5
Jabbar,Ahmad V.	987987987	1969-03-29	980 Dallas,Houston,TX	4
Borg,James E.	888665555	1937-11-10	450 Stone,Houston,TX	1

DEPARTMENT

DNAME	<u>DNUMBER</u>	DMGRSSN
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

<u>DNUMBER</u>	<u>DLOCATION</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Priyanka Dhanraj

WORKS_ON

<u>SSN</u>	<u>PNUMBER</u>	HOURS
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0

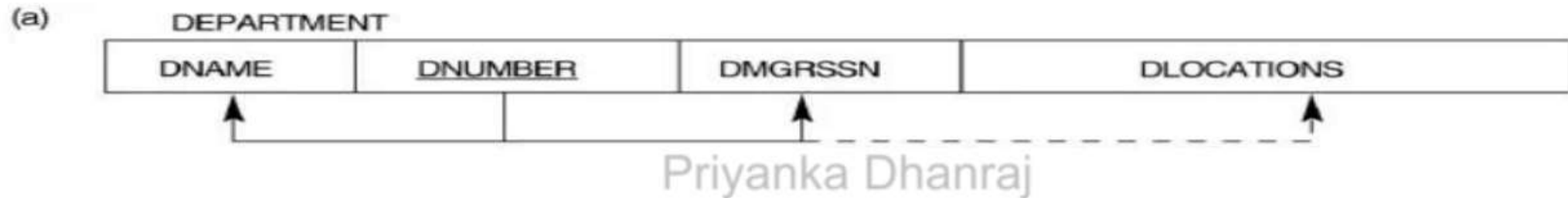
PROJECT

PNAME	<u>PNUMBER</u>	PLOCATION	DNUM
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

Normalization in DBMS :1NF

Normalization into 1NF:

- (a) A relation schema that is not in 1NF.
- (b) Example state of relation DEPARTMENT.
- (c) 1NF version of same relation with redundancy.



(b)

DEPARTMENT			
DNAME	<u>DNUMBER</u>	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT			
DNAME	<u>DNUMBER</u>	DMGRSSN	<u>DLOCATION</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Normalization in DBMS :1NF

- **1NF also Disallows Nested relations:** Normalizing nested relations into 1NF.(a) Schema of the EMP_PROJ relation with a “nested relation” attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple.(c) Decomposition of the EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key

(a)

EMP_PROJ			
SSN	ENAME	PROJS	
		PNUMBER	HOURS

(b)

EMP_PROJ			
SSN	ENAME	PNUMBER	HOURS
123456789	Smith,John B.	1	32.5
		2	7.5
666884444	Narayan,Ramesh K.	3	40.0
453453453	English,Joyce A.	1	20.0
		2	20.0

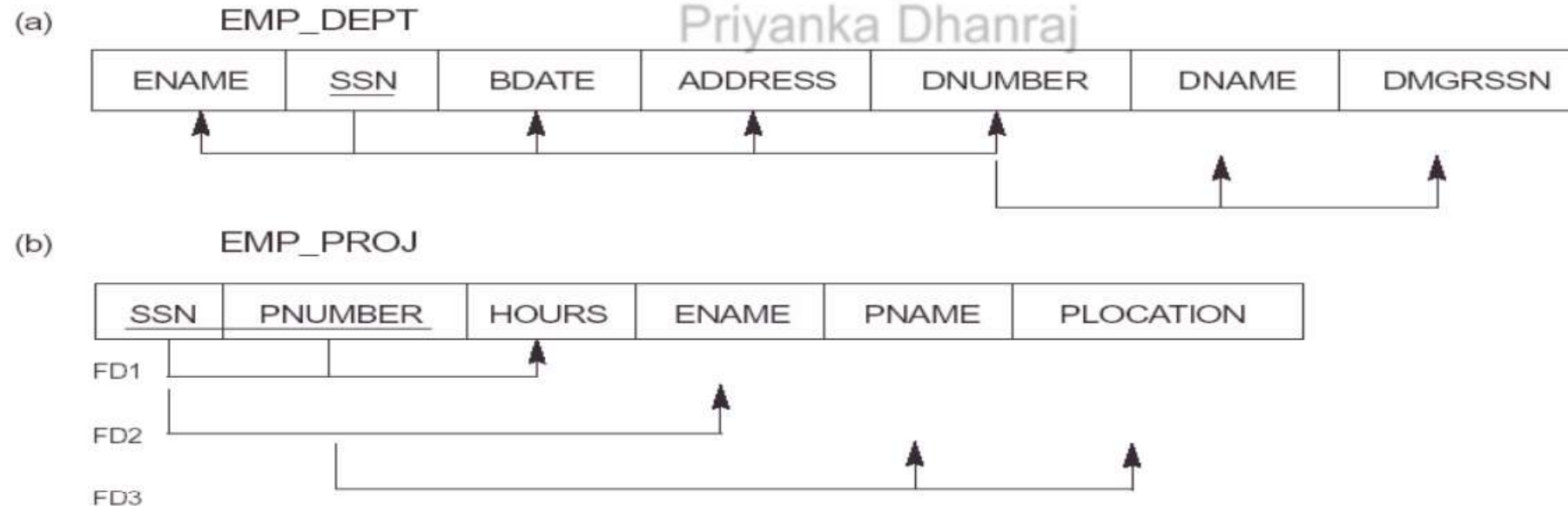
(c)

EMP_PROJ1	
<u>SSN</u>	ENAME

EMP_PROJ2		
<u>SSN</u>	<u>PNUMBER</u>	HOURS

Redundant Information in Tuples and Update Anomalies

- Mixing attributes of multiple entities may cause problems
- Information is stored redundantly wasting storage
- Problems with update anomalies: Insertion anomalies, Deletion anomalies, Modification anomalies



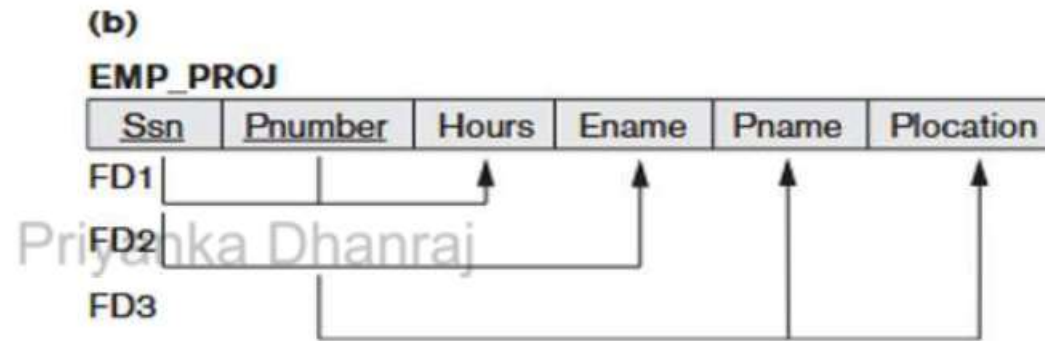
Redundant Information in Tuples and Update Anomalies

Consider the relation schema EMP_PROJ in Figure from the semantics of the attributes and the relation, we know that the following functional dependencies should hold:

a. $Ssn \rightarrow Ename$

b. $Pnumber \rightarrow \{Pname, Plocation\}$

c. $\{Ssn, Pnumber\} \rightarrow Hours$



These functional dependencies specify that (a) the value of an employee's Social Security number (Ssn) uniquely determines the employee name (Ename), (b) the value of a project's number (Pnumber) uniquely determines the project name (Pname) and location (Plocation), and (c) a combination of Ssn and Pnumber values uniquely determines the number of hours the employee currently works on the project per week (Hours). Alternatively, we say that Ename is functionally determined by (or functionally dependent on) Ssn, or given a value of Ssn, we know the value of Ename, and so on.

EXAMPLE OF AN ANOMALY

Consider the relation:

EMP_PROJ (Emp#, Proj#, Ename, Pname, No_hours)

- **Update Anomaly**

- Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1

- **Insert Anomaly**

- Cannot insert a project unless an employee is assigned to .
- Inversely- Cannot insert an employee unless he/she is assigned to a project.

- **Delete Anomaly**

- When a project is deleted, it will result in deleting all the employees who work on that project. Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Design a schema that does not suffer from the insertion, deletion and update anomalies.

If there are any present, then note them so that applications can be made to take them into account

EMP_DEPT

ENAME	<u>SSN</u>	BDATE	ADDRESS	DNUMBER	DNAME	DMGRSSN
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Priyanka Dhanraj

EMP_PROJ

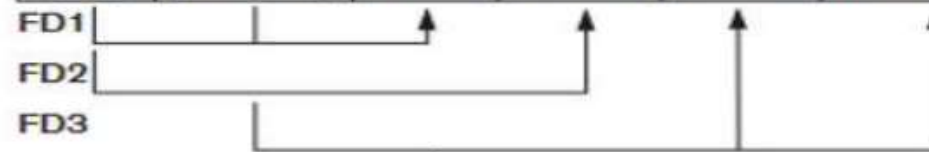
<u>SSN</u>	<u>PNUMBER</u>	HOURS	ENAME	PNAME	PLOCATION
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford

Normalization in DBMS : 2 NF and 3NF

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

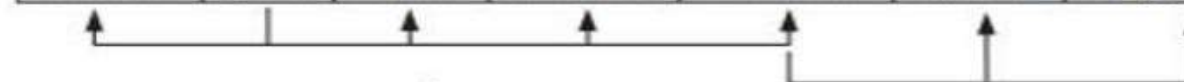
<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



(b)

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	<u>Dnumber</u>	Dname	Dmgr_ssn
-------	------------	-------	---------	----------------	-------	----------



3NF Normalization

ED1

Ename	<u>Ssn</u>	Bdate	Address	<u>Dnumber</u>
-------	------------	-------	---------	----------------



ED2

<u>Dnumber</u>	Dname	Dmgr_ssn
----------------	-------	----------



Null Values in Tuples and Spurious Tuples

- Relations should be designed such that their tuples will have as few NULL values as possible. Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- The relations should be designed to satisfy the lossless join condition. No spurious (Less/ Extra) tuples should be generated by doing a natural-join of any relations.

Inclusion dependency

- It is a statement in which **a few of the columns of a relation are in the other columns**. Typically, inclusion dependency has a very insignificant influence on the design of a database. For example, a foreign key is inclusion dependency.



Explanation of inclusion dependency

- Let there be the relations S and R that was acquired by the translation of entity sets so that every S entity is an R entity.
- Inclusion dependency occurs when the relation obtained by the projection of S on the key attributes is the same as that retrieved from R. There should be no splitting up of the collections of attributes that take part in the inclusion dependency.
- Generally, the inclusion dependencies are based on keys.

Alternative approaches to database design : Domain Key Normal Form (DKNF)

- In domain key normal form there should be no constraints except for the key constraint and domain constraint.
- Key constraint: Constraint in general means some kind of restriction. In a relational schema, a key constraint means that :
 1. The key value is not null
 2. It should uniquely identify two tuples
- Domain constraint :. To better understand this consider a voter table with attributes such as name, age, and voter_id. The domain constraint here can be that the age must be a positive integer and greater than or equal to 18. Here we specify the legal values that can be allowed in the age field. If someone tries to insert -3 or 14 as age, it will be a violation of the domain constraint and an error will be thrown.
- A table is in domain key normal form if:
 1. The table has domain and key constraint
 2. It should not have any other general constraint

Alternative approaches to database design : Domain Key Normal Form (DKNF)

roll_no	marks	remarks
100	96	excellent
101	92	excellent
102	80	very good
103	50	average
104	40	satisfactory

Priyanka Dhanraj

roll_no	marks
100	96
101	92
102	80
103	50
104	40

marks	remarks
90	excellent
70	very good
40	average
30	satisfactory
<30	failed

Alternative approaches to database design : Domain Key Normal Form (DKNF)

- We see that all the roll_no are unique and it uniquely identifies two tuples.
- Similarly, we can argue that the domain constraint is also not violated considering the domains of roll_no and marks to be positive integers and that of remarks to be a set of characters.
- However, can we claim that there are no general constraints here? Can we guarantee that there are no insertion and deletion anomalies here?
- If we try to add only marks as 30 or less in the marks column and “failed” in the remarks column without adding anything in the roll_no column(since we assumed no one failed), we will violate the key constraint since the roll_no field will be left null.
- If we delete the tuple with roll_no = 103, we lose data. Why? Deleting roll_no 103 will also delete the values 50 and average from the marks and remarks column.
- Both the tables are now free from insertion and deletion anomalies. We can add any number of (roll_no, marks) pairs or any number of (marks, remarks) pairs. Similarly, we can delete any number of (roll_no, marks) pairs or (marks, remarks) pairs from the second table without any loss of information.
- By doing so, we guarantee that there are no insertion and deletion anomalies, also the table has the key and domain constraints intact without any other general constraint. This is how we claim the tables to be in domain key normal form.